*** Query**
[SQL 2023] 7.17 <query expression>

WITH aka Common Table Expression (CTE)
Wiki: https://en.wikipedia.org/wiki/Hierarchical_and_recursive_queries_in_SQL#Common_table_expression
Subquery assigned to a name

```
-- subquery
SELECT x FROM (SELECT * FROM mytable)

-- same CTE
WITH t AS (SELECT * FROM mytable)
  SELCT x FROM t
```

```
-- 3 basic forms
--
SELECT 1;                          -- 1) SELECT
expression
VALUES (1, 'a'), (2, 'b');         -- 2) VALUES
VALUES 1, 2;                       -- ERROR Need ()
TABLE t;                           -- 3) explicit TABLE
```

```
-- e.g.

-- SELECT
> SELECT 1;
 ?column?
----------
        1

> SELECT 1 AS x;
   x
--------
   1


-- VALUES
> VALUES (1, 'a'), (2, 'b');
 column1 | column2
---------+---------
       1 | a
       2 | b


-- TABLE
> SELECT x, y INTO t FROM (VALUES (1, 'a'), (2, 'b') AS
_(x, y));
> TABLE t;
```

```
 column1 | column2
─────────┼─────────
       1 | a
       2 | b
```

```
-- UNION/INTERSECTION/EXCEPT
--
-- can combine with any
VALUES (1, 'a'), (2, 'b')
  UNION (VALUES ...)          -- with VALUES
  UNION (SELECT ...)          -- with SELECT
  UNION (TABLE t)             -- with TABLE
  ...

  -- t1 ∪ t2
  UNION ...                   -- no duplicates by
default
  UNION DISTINCT ...          -- same explicit
  UNION ALL ...               -- keeps duplicates

  -- t1 ∩ t2
  INTERSECT ...               -- no duplicates by
default
  INTERSECT DISTINCT ...      -- same explicit
  INTERSECT ALL ...           -- keep duplicates
                              -- minimum of both tables
i.e. {a,a,a,b} INTERSECT ALL {a,a} = {a, a}

  -- t1 / t2
  EXCEPT ...                  -- no duplicates by
default
  EXCEPT DISTINCT ...         -- same explicit
  EXCEPT ALL ...



---- WITH aka CTE
WITH x AS (SELECT 1, 'a')
  SELECT * FROM x;
WITH x AS (SELECT 1, 'a')         -- same
  TABLE x;

WITH
  x AS (SELECT 1, 'a')                -- basic
  x(a, b) AS (SELECT  1, 'a')         -- specify column
name
  x AS (SELECT 1 AS a, 'a' AS b)      -- same

  -- multiple
  x2 AS (SELECT ...),
  x3 AS (SELECT ...),
  x4 AS (SELECT ...)
```

```
  -- WITH body: can be any query
  -- i.e.
  x AS (SELECT ...)                    -- 1) SELECT
  x AS (VALUES ...)                    -- 2) VALUES
  x AS (TABLE t)                       -- 3) TABLE
  x AS (WITH y AS (...) TABLE y)       -- can nest WITHs


-- WITH RECURSIVE: recursive reference of itself
-- e.g. factorial
WITH RECURSIVE factorial (n, result) AS (
  SELECT 0, 1
    UNION ALL
  SELECT n, (n+1)*result FROM factorial WHERE n <= 5
)
TABLE factorial
```

```
-- Output
0, 1
1, 1
2, 2
3, 6
4, 24
5, 120
```

*** SELECT**

SELECT

Select: https://en.wikipedia.org/wiki/Select_(SQL)
[SQL 2023] 7.11 <query specification>

```
-- Basic
SELECT 1                               -- 1
SELECT a FROM mytable
SELECT mytable.a FROM mytable

-- columns
SELECT
  1,                           -- literal
  a,                           -- column a
  mytable.a,                   -- same explicit
  a, b,                        -- multiple
  a AS name,                   -- alias
  a + b,                       -- expression

  -- subquery
  (SELECT 1),                  -- subquery with EXACTLY
one value
  (SELECT * FROM t),           -- ERROR Only one column
  (SELECT id FROM t),          -- RUNTIME ERROR if more
than 1 row
```

```
  -- composite type obj = { id: 1 }
  obj.id,                      -- ERROR. "obj" is treated
as table name
  (obj).id,                    -- OK
  (mytable).obj.id,            -- same
  (mytable).obj.child.id       -- nested

  -- wildcard
  *,
  (mytable).*,                 -- same
  a, b, (mytable).*,           -- with normal columns
  (obj).*,                     -- expand obj
  (mytable).obj.*,             -- same
  obj.child.*,                 -- same
```

```
-- e.g. select t2 columns only
SELECT t2.* FROM t1 INNER JOIN t2 on t1.id = t2.id;
```

```
  ---- FROM
  FROM mytable2
  FROM (SELECT ...) t     -- subquery
  FROM (SELECT ...)       -- ERROR subquery must have
alias

  -- alias
  FROM mytable2 t         -- alias t=mytable2
  FROM mytable2 AS t      -- same explicity
  FROM mytable2 t(a, b)   -- alias a=mytable2.column1
                          -- alias b=mytable2.column2
  FROM mytable2 t(a)      -- ERROR


  -- Multiple
  FROM mytable, mytable2              -- multiple tables
                                      -- same as cross
join i.e. Cartesian product
  FROM mytable CROSS JOIN mytable2    -- same


-- WHERE
SELECT ... FROM mytable WHERE a = 1
SELECT
  FROM mytable

  -- WHERE
  WHERE NULL              -- empty table
  WHERE a = 1 AND b = 2
```

```
  -- ORDER BY
  -- [SQL 2023] 10.10 <sort specification list>
  --
  ORDER BY x          -- ascending
  ORDER BY x, y       -- multiple
                      -- order by x, or y if x is equal
  ORDER BY x ASC
  ORDER BY x DESC


  -- LIMIT
  LIMIT 10
  LIMIT ALL           -- no limit


  -- OFFSET: skip
  -- FETCH FIRST: drop
  --
  OFFSET 5                        -- e.g. drop first 5
rows

  FETCH FIRST ROW ONLY
  FETCH FIRST ROWS ONLY           -- same
  FETCH NEXT ROWS ONLY            -- same
  FETCH FIRST 1 ROWS ONLY         -- same
  FETCH FIRST 5 ROWS ONLY
  FETCH FIRST 1 PERCENT ROWS ONLY      -- by ratio. Not
always supported.

  -- WITH TIES: include duplicates of the last row
  ORDER BY n FETCH FIRST 5 ROWS WITH TIES
  FETCH FIRST 5 ROWS WITH TIES                 -- ERROR.
Requires ORDER BY
```

```
-- e.g. if t = 1,1,2,2,3,3,3,3,3,4
--
-- ONLY
SELECT * FROM t ORDER BY x FETCH FIRST 5 ROWS ONLY
    -- exactly 5 rows
1
1
2
2
3

-- WITH TIES
SELECT * FROM t ORDER BY x FETCH FIRST 3 ROWS WITH
TIES    -- 8 rows
1
    -- includes duplicates of the last row
1
2
2
```

```
3
3
3
3
```

```
  -- TABLESAMPLE: random sample
  TABLESAMPLE SYSTEM(10)          -- 10%
  TABLESAMPLE BERNOULLI(10)
```

*** GROUP BY**

```
---- Grouping
SELECT x, SUM(y) FROM mytable GROUP BY x
SELECT x, y, SUM(z) FROM mytable GROUP BY x, y     --
multiple
SELECT x, y FROM mytable GROUP BY x                --
BAD. non-grouped y is not an aggregate

-- WHERE vs HAVING
--
-- WHERE:  filter applied before grouping
-- HAVING: filter applied after grouping
SELECT x, SUM(y) FROM mytable
  WHERE y > 0                      -- group only rows
with y > 0
  GROUP BY x
SELECT x, SUM(y) FROM mytable
  GROUP BY x
  HAVING SUM(y) > 0                -- only groups with
SUM(y) > 0
SELECT x, SUM(y) FROM mytable      -- combined
  WHERE y > 0
  GROUP BY x
  HAVING SUM(y) > 10


-- TODO
-- GROUPING SET
-- ROLLUP
```

*** Subqueries/CTE**
[SQL 2023] 7.19 <subquery>

Subquery
Nested SELECT statements

aka a **virtual table** created by the subquery

```
---- Subquery
--
--
SELECT x FROM
  (SELECT * ...)
SELECT t.x FROM
  (SELECT * ...) t                -- with name t
SELECT * FROM mytable
  WHERE x < (SELECT AVG(y) ...)   -- in WHERE

-- any table expression
SELECT * FROM
  (SELECT * ...)                  -- a) SELECT ...
  (VALUES (1, 'a'), (2, 'b'))     -- b) VALUES ...
  (TABLE t)                       -- c) explicit TABLE
```

*** JOIN**
JOIN
Wiki: https://en.wikipedia.org/wiki/Join_(SQL)
3 types of joins
- **CROSS aka Cartesian product**: all possible combinations

- **INNER**: join only where **both** tables have an entry

- **OUTER (LEFT, RIGHT, FULL)**: join for each entry

LEFT	join for each entry in left table. NULL for right entry if no match.
RIGHT	join for each entry in right table. NULL for left entry if no match.
FULL	left and right combined. i.e. join for each entry if either left or right exists. NULL entry if one does not exist.

e.g.
Data

```
t1          t2
id value    id  value
2  'a'      3   'A'
4  'b'      6   'B'
6  'c'      9   'C'
8  'd'      12  'D'
10 'e'
12 'f'
```

Joins

```
-- Cross join
-- SELECT * from t1 CROSS JOIN t2 ON t1.id = t2.id
```

```
t1.id    t1.value     t2.id    t2.value
2        'a'          3        'A'
2        'a'          6        'B'
2        'a'          9        'C'
2        'a'          12       'D'
4        'a'          3        'A'
...
2 'a' 6 'B'
2 'a' 9 'C'
2 'a' 12 'D'
4 'b' 3 'A'
...


-- INNER JOIN
--
-- SELECT * from t1 JOIN t2 ON t1.id = t2.id
t1.id    t1.value     t2.id    t2.value
6        'c'          3        'C'
12       'f'          12       'D'


-- LEFT JOIN
--
-- SELECT * from t1 LEFT JOIN t2 ON t1.id = t2.id
t1.id    t1.value     t2.id    t2.value
2        'a'          NULL     NULL
4        'b'          NULL     NULL
6        'c'          6        'B'
8        'd'          NULL     NULL
10       'e'          NULL     NULL
12       'f'          12       'D'


-- RIGHT JOIN
--
-- SELECT * from t1 RIGHT JOIN t2 ON t1.id = t2.id
t1.id    t1.value     t2.id    t2.value
NULL     NULL         3        'A'
6        'c'          6        'B'
NULL     NULL         9        'C'
12       'f'          12       'D'


-- FULL JOIN
--
-- SELECT * from t1 FULL JOIN t2 ON t1.id = t2.id
t1.id    t1.value     t2.id    t2.value
2        'a'          NULL     NULL
NULL     NULL         3        'A'
4        'b'          NULL     NULL
6        'c'          6        'B'
8        'd'          NULL     NULL
NULL     NULL         9        'C'
```

```
10      'e'          NULL     NULL
12      'f'          12       'D'
```

```
-- Cross
--
SELECT * FROM t1 CROSS JOIN t2
SELECT * FROM t1,
t2                                    -- implicit
SELECT * FROM t1 INNER JOIN ON
TRUE                 -- same with inner join

-- Inner
SELECT * FROM t1 JOIN t2 ON t1.id = t2.id
SELECT * FROM t1 INNER JOIN t2 ON t1.id =
t2.id            -- same
SELECT * FROM t1, t2 ON t1.id =
t2.id                    -- same with cross join

-- Outer
SELECT * FROM t1 LEFT OUTER JOIN t2 ON t1.id = t2.id
SELECT * FROM t1 LEFT JOIN t2 ON t1.id = t2.id
    -- same
SELECT * FROM t1 RIGHT OUTER JOIN t2 ON t1.id = t2.id
SELECT * FROM t1 RIGHT JOIN t2 ON t1.id = t2.id
    -- same
SELECT * FROM t1 FULL OUTER JOIN t2 ON t1.id = t2.id

SELECT * FROM t1 FULL JOIN t2 ON t1.id = t2.id
    -- same
```

*** LATERAL JOIN**

[SQL 2023] 7.6 <table reference>: <lateral derived table>
[PostgreSQL] LATERAL subqueries: https://www.postgresql.org/docs/current/queries-table-expressions.html#QUERIES-LATERAL
[SO] INNER JOIN vs LATERAL JOIN: https://stackoverflow.com/questions/49624547/postgresql-lateral-vs-inner-join

LATERAL subquery
Subquery with reference to row in **outer (parent) scope.**

LATERAL JOIN vs regular JOIN
Different possible **execution plans**.
In particular, LATERAL JOIN restricts the **JOIN order**, since the **parent** always has to be **first**.

e.g.

JOIN	**LATERAL JOIN**
Subquery	
SELECT ...	SELECT ...
SELECT ...	

```
      FROM t1                    FROM t1                          (
      INNER JOIN (               INNER JOIN LATERAL (
SELECT t2.value
        SELECT id, value          SELECT t2.value
FROM t2.value
        FROM t2                   FROM t2
WHERE t1.id = t2.id
      ) t ON t1.id = t.id         WHERE t1.id = t2.id              )
                                ) t ON TRUE                      FROM
t1
```

Execution plan for LATERAL JOIN: t1 must be **before** t2.

```
Nested Loop
  -> Seq Scan on t1
  -> Index Scan on t2
     Index Cond: t1.id = t2.id

Hash Join
  Hash Cond: t1.id = t2.id
  -> Seq Scan on t1
  -> Hash
     -> Seq Scan on t2
```

Other execution plans for normal JOIN

```
Nested Loop
  -> Seq Scan on t2                    -- t2 before t1
  -> Index Scan on t1
     Index Cond: t1.id = t2.id

Merge Join
  -> Index Scan on t1
  -> Index Scan on t2
     Index Cond: t1.id = t2.id

Hash Join
  Hash Cond: t1.id = t2.id             -- t2 before t1
  -> Seq Scan on t2
  -> Hash
     -> Seq Scan on t1
```

*** JOIN Subquery collapse**

JOIN collapse

Equivalent forms of JOINs and subquery JOINs

Relationship	Original Collapsed
t1 INNER (t2 INNER t3) = t1 INNER t2 INNER t3	SELECT ... FROM t1

```
 FROM t1
                          INNER JOIN (
 INNER JOIN t2 ON
                            SELECT t2.id, ...
   t1.id = t2.id
                            FROM t2
 INNER JOIN t3 ON
                            INNER JOIN t3 ON
   t2.key = t2.key
                            t2.key = t3.key

                          ) t ON
                           t1.id = t.id


t1 INNER (t2 LEFT t3)     SELECT ...
 SELECT
 = t1 LEFT t2 LEFT t3     FROM t1
 FROM t1
                          INNER JOIN (
 INNER JOIN t2 ON
                           SELECT t2.id, ...
   t2.id = t3.id
                           FROM t2
 LEFT OUTER JOIN t3 ON
                           LEFT OUTER JOIN t3 ON
   t2.key = t3.key
                            t2.key = t3.key

                          ) t ON
                           t1.id = t.id



t1 LEFT (t2 INNER t3)     SELECT ...
 NA?
 = NA                     FROM t1

                          LEFT OUTER JOIN (

                           SELECT t2.id, ...

                           FROM t2

                           LEFT OUTER JOIN t3 ON

                            t2.key = t3.key

                          ) t ON
                           t1.id = t.id



t1 LEFT (t2 LEFT t3)      SELECT ...
```

```
    SELECT
    = t1 LEFT t2 LEFT t3      FROM t1
    FROM t1
                              LEFT OUTER JOIN (
    LEFT OUTER JOIN t2
                               SELECT t2.id, ...
      t2.id = t3.id
                               FROM t2
    LEFT OUTER JOIN t3 ON
                               LEFT OUTER JOIN t3 ON
      t2.key = t3.key
                                t2.key = t3.key

                              ) t ON
                               t1.id = t.id
```

***) JOIN associativity**

t1 LEFT (t2 INNER t3)
= t1 LEFT t2 LEFT t3

```
SELECT ...
FROM t1
LEFT OUTER JOIN (
  SELECT t2.id
  FROM t2
  INNER JOIN t3 ON
   t2.key = t3.key
) t ON
 t.id = t.id


SELECT ...                    -- Same collapsed
FROM t1
LEFT OUTER JOIN t2 ON
 t1.id = t2.id
LEFT OUTER JOIN t3 ON
 t2.key = t3.key


SELECT ...                    -- BAD: NOT same
FROM t1
LEFT OUTER JOIN t2 ON
 t1.id = t2.id
INNER JOIN t3 ON              -- BAD. This is INNER JOIN
on t1+t3, NOT on t2+t3
 t2.key = t3.key              -- i.e. becomes same as
                              --
                              --    FROM t1
                              --    INNER JOIN t2 ON
                              --      t1.id = t2.id
                              --    INNER JOIN t3 ON
                              --      t2.key = t3.key
```

==========
======
== Windows ==

[SQL 2023] 4.17.15 Windowed tables
[SQL 2023] 4.18.3 Window functions
[SQL 2023] 6.10 <window function>
[SQL 2023] 7.15 <window clause>
PostgreSQL window function: https://www.postgresql.org/docs/current/sql-expressions.html#SYNTAX-WINDOW-FUNCTIONS

Window
Window function
Partitioning like **GROUP BY**, but retain individual rows.

Rows are collected into a **WINDOW**.
- **PARTITION BY**: each window is further split into **partitions** by the **partition key**.
A row's window only contains values in the **same partition**.

- **ORDER BY**: each window is further split into **peer groups** by the **order key**.
Each window is constructed by adding peer groups in the order of the order key.
A row's window only contains **peer groups <= the order key** of the row.

Thus, a row's window contains
- rows in the **same partition**.
- rows in the **peer groups <=** the row's peer group.
Peer groups are **ordered** by the order key.

GOTCHA: rows in the same partition may NOT have the same window.
The window is the same if and only if they have the same **partition key** and **order key**.

e.g. for numbers n = 1,2,3...,30
PARTITION BY n % 2
ORDER BY n % 3

values	**partition value**	**peer value**	**peer group**	**window**
6,12,24,30,18	n % 2 = 0	n % 3 = 0	{6,12,24,30,18}	{6,12,24,30,18} peer groups <= 0
4,10,22,28	n % 2 = 0	n % 3 = 1	{4,10,22,28}	{6,12,24,30,18,4,10,22,28} peer groups <= 1
26,20,8,14,2	n % 2 = 0	n % 3 = 2	{26,20,8,14,2}	{6,12,24,30,18,4,10,22,28, 26,20,8,14,2} peer groups <= 2
21,15,3,27,9,1	n % 2 = 1	n % 3 = 0	{21,15,3,27,9,1}	{21,15,3,27,9,1} peer groups <= 0
7,13,19,25,17	n % 2 = 1	n % 3 = 1	{7,13,19,25,17}	{21,15,3,27,9,1,7,13,19,25,17} peer groups <= 1
29,23,5,11	n % 2 = 1	n % 3 = 2	{29,23,5,11}	{21,15,3,27,9,17,13,19,25,17,29,23,5,11} peer groups <= 2

Window: all

```
SELECT
  n,
  array_agg(n) OVER() w
FROM generate_series(1, 30) _(n)
    -- NOTE: PostgreSQL function
ORDER BY n;

 n  |                                                w
----+-----------------------------------------------------------------
-----------------------------------

  0 |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21
,22,23,24,25,26,27,28,29,30}
  1 |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21
,22,23,24,25,26,27,28,29,30}
  2 |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21
,22,23,24,25,26,27,28,29,30}
  3 |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21
,22,23,24,25,26,27,28,29,30}
  4 |
{0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21
,22,23,24,25,26,27,28,29,30}
...
```

Window: PARTITION BY

```
SELECT
  n,
  array_agg(n) OVER(PARTITION BY n % 2) w
FROM generate_series(1, 30) _(n)
    -- NOTE: PostgreSQL function
ORDER BY n;

 n  |                      w
----+-----------------------------------------------
  1 |
{1,17,3,25,5,19,7,29,9,21,11,27,13,23,15}          --
partition %2 = 1
  2 |
{30,2,4,6,8,10,12,14,16,18,20,22,24,26,28}     --
partition %2 = 0
  3 | {1,17,3,25,5,19,7,29,9,21,11,27,13,23,15}
  4 | {30,2,4,6,8,10,12,14,16,18,20,22,24,26,28}
  5 | {1,17,3,25,5,19,7,29,9,21,11,27,13,23,15}
  6 | {30,2,4,6,8,10,12,14,16,18,20,22,24,26,28}
  7 | {1,17,3,25,5,19,7,29,9,21,11,27,13,23,15}
  8 | {30,2,4,6,8,10,12,14,16,18,20,22,24,26,28}
....
```

Window: ORDER BY

```
SELECT
  n,
  array_agg(n) OVER(ORDER BY n % 3) w
FROM generate_series(1, 30) _(n)
    -- NOTE: PostgreSQL function
ORDER BY n;
....
(30 rows)

 n  |                                              w
----+-------------------------------------------------
-------------------------------
  1 |
{21,12,27,18,15,6,3,24,9,30,16,4,7,10,13,1,19,22,25,28}
                               -- peer groups <= 1
  2 |
{21,12,27,18,15,6,3,24,9,30,16,4,7,10,13,1,19,22,25,28,1
1,2,23,8,29,26,17,14,5,20}         -- peer groups <= 2
  3 | {21,12,27,18,15,6,3,24,9,30}
                                      -- peer groups <=
0
  4 |
{21,12,27,18,15,6,3,24,9,30,16,4,7,10,13,1,19,22,25,28}
  5 |
{21,12,27,18,15,6,3,24,9,30,16,4,7,10,13,1,19,22,25,28,1
1,2,23,8,29,26,17,14,5,20}
  6 | {21,12,27,18,15,6,3,24,9,30}
  7 |
{21,12,27,18,15,6,3,24,9,30,16,4,7,10,13,1,19,22,25,28}
  8 |
{21,12,27,18,15,6,3,24,9,30,16,4,7,10,13,1,19,22,25,28,1
1,2,23,8,29,26,17,14,5,20}
  9 | {21,12,27,18,15,6,3,24,9,30}
 10 |
{21,12,27,18,15,6,3,24,9,30,16,4,7,10,13,1,19,22,25,28}
 11 |
{21,12,27,18,15,6,3,24,9,30,16,4,7,10,13,1,19,22,25,28,1
1,2,23,8,29,26,17,14,5,20}
....
```

Ranks/Row numbers
Position in partition

row number	absolution position of **row** in partition
rank	absolution position of peer group in partition i.e. absolution position of first row in peer group
dense rank	relative position of **peer group** in partition, if ordered

i.e. 1st peer group dense rank=1
2nd peer group dense rank=2
always 1 if not ordered

e.g. partition + order

n = 21
window {21,15,3,27,9}
peer group {21,15,3,27,9}
row number = position of 1 = 1
dense rank = position of peer group = 1
rank = first row position of peer group = 1

n = 3
window {21,15,3,27,9}
peer group {21,15,3,27,9}
row number = position of 3 = 3
dense rank = position of peer group = 1
rank = first row position of peer group = 1

n = 7
window {21,15,3,27,9,1,7,13,19,25}
peer group {1,7,13,19,25}
row n umber = position of 1 = 7
dense rank = position of peer group = 2
rank = first row position of peer group = 6

n = 13
window {21,15,3,27,9,1,7,13,19,25}
peer group {1,7,13,19,25}
row n umber = position in 13 = 9
dense rank = position of peer group = 2
rank = row position of peer group = 6

```
SELECT
 n,
 array_agg(n) OVER w AS window,
 ROW_NUMBER() OVER w,
 RANK() OVER w,
 DENSE_RANK() OVER w
FROM generate_series(1, 30)
_(n)                        -- NOTE: PostgreSQL function
WINDOW w AS (PARTITION BY n % 2 ORDER BY n % 3)
ORDER BY n;

 n  |              window                          |
row_number | rank | dense_rank
════╪══════════════════════════════════════════════╪═══════
═══════╪══════╪════════════        --  partition value
    peer value  dense rank   peer group
  1 | {21,15,3,27,9,1,7,13,19,25}                  |
6 |    6 |          2         --  n % 2 = 1            n
% 3 = 1  2            {1,7,13,19,25}
  2 | {6,12,24,30,18,16,4,10,22,28,26,20,8,14,2} |
15 |   11 |          3            --  n % 2 = 0
    n % 3 = 2    3            {26,20,8,14,2}
```

```
  3 | {21,15,3,27,9}                                |
3 |    1 |          1          --  n % 2 = 1            n
% 3 = 0  1            {21,15,3,27,9}
  4 | {6,12,24,30,18,16,4,10,22,28}                 |
7 |    6 |          2          --  n % 2 = 0            n
% 3 = 1  2            {16,4,10,22,28}
  5 | {21,15,3,27,9,1,7,13,19,25,17,29,23,5,11}     |
14 |   11 |           3             --  n % 2 = 1
    n % 3 = 2    3            {17,29,23,5,11}
  6 | {6,12,24,30,18}                               |
1 |    1 |          1          --  n % 2 = 0            n
% 3 = 0  1            {6,12,24,30,18}
  7 | {21,15,3,27,9,1,7,13,19,25}                   |
7 |    6 |          2
  8 | {6,12,24,30,18,16,4,10,22,28,26,20,8,14,2}    |
13 |   11 |           3
  9 | {21,15,3,27,9}                                |
5 |    1 |          1
 10 | {6,12,24,30,18,16,4,10,22,28}                 |
8 |    6 |          2
 11 | {21,15,3,27,9,1,7,13,19,25,17,29,23,5,11}     |
15 |   11 |           3
 12 | {6,12,24,30,18}                               |
2 |    1 |          1
 13 | {21,15,3,27,9,1,7,13,19,25}                   |
8 |    6 |          2
 14 | {6,12,24,30,18,16,4,10,22,28,26,20,8,14,2}    |
14 |   11 |           3
 15 | {21,15,3,27,9}                                |
2 |    1 |          1
 16 | {6,12,24,30,18,16,4,10,22,28}                 |
6 |    6 |          2
 17 | {21,15,3,27,9,1,7,13,19,25,17,29,23,5,11}     |
11 |   11 |           3
 18 | {6,12,24,30,18}                               |
5 |    1 |          1
 19 | {21,15,3,27,9,1,7,13,19,25}                   |
9 |    6 |          2
 20 | {6,12,24,30,18,16,4,10,22,28,26,20,8,14,2}    |
12 |   11 |           3
 21 | {21,15,3,27,9}                                |
1 |    1 |          1
 22 | {6,12,24,30,18,16,4,10,22,28}                 |
9 |    6 |          2
 23 | {21,15,3,27,9,1,7,13,19,25,17,29,23,5,11}     |
13 |   11 |           3
 24 | {6,12,24,30,18}                               |
3 |    1 |          1
 25 | {21,15,3,27,9,1,7,13,19,25}                   |
10 |    6 |          2
 26 | {6,12,24,30,18,16,4,10,22,28,26,20,8,14,2}    |
11 |   11 |           3
 27 | {21,15,3,27,9}                                |
4 |    1 |          1
```

```
 28 | {6,12,24,30,18,16,4,10,22,28}               |
10 |    6 |          2
 29 | {21,15,3,27,9,1,7,13,19,25,17,29,23,5,11}  |
12 |   11 |          3
 30 | {6,12,24,30,18}                            |
4 |    1 |          1

(30 rows)
```

e.g. partition only

```
SELECT
 n,
 array_agg(n) OVER w AS window,
 ROW_NUMBER() OVER w,
 RANK() OVER w,
 DENSE_RANK() OVER w
FROM t
WINDOW w AS (PARTITION BY n % 3)
ORDER BY n;

 n  |            window             | row_number | rank |
dense_rank
════╪═══════════════════════════════╪════════════╪══════╪
═══════════
  1 | {16,4,7,10,13,1,19,22,25,28} |          6 |    1 |
1
  2 | {11,2,23,8,29,26,17,14,5,20} |          2 |    1 |
1
  3 | {21,12,27,18,15,6,3,24,9,30} |          7 |    1 |
1
  4 | {16,4,7,10,13,1,19,22,25,28} |          2 |    1 |
1
  5 | {11,2,23,8,29,26,17,14,5,20} |          9 |    1 |
1
...
```

Modes

Examples
e.g. **empty**
Only one partition i.e. **whole table**
Only one window

```
SELECT
  n, array_agg(n) AS window
  OVER()
FROM t
ORDER BY n;
```

```
 n  |      window
  1 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}     -- window = whole table
  2 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
  3 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
  4 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
  5 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
  6 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
  7 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
  8 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
  9 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 10 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 11 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 12 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 13 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 14 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 15 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 16 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 17 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
2,23,24,25,26,27,...}
 18 |
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,2
```

```
2,23,24,25,26,27,...}
```

e.g. **ORDER BY** only
ORDER BY n: one partition i.e. **whole table** + distinct peers

```
SELECT
  n, array_agg(n) AS window
  OVER(ORDER BY n)
FROM t
ORDER BY n;

 n  |                   window
════╪═══════════════════════════════════════
  1 | {1}
  2 | {1,2}
  3 | {1,2,3}
  4 | {1,2,3,4}
  5 | {1,2,3,4,5}
  6 | {1,2,3,4,5,6}
  7 | {1,2,3,4,5,6,7}
  8 | {1,2,3,4,5,6,7,8}
  9 | {1,2,3,4,5,6,7,8,9}
 10 | {1,2,3,4,5,6,7,8,9,10}
 11 | {1,2,3,4,5,6,7,8,9,10,11}
...
```

ORDER BY n % 5: one partition (i.e. whole table) + 5 peer groups

```
SELECT
  n, array_agg(n)
  OVER(ORDER BY n % 5) AS window
FROM t
ORDER BY n;

 n  |                                        window
════╪═══════════════════════════════════════════════════════════════
═══════════════════════════════════
  1 | {15,10,25,20,5,30,16,6,11,1,21,26}
                                                    -- peer
group n % 5 = 1 {16,6,11,1,21,26}
  2 |
{15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27}
                                         -- peer group n %
5 = 2 {12,7,17,22,2,27}
  3 |
{15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27,18,8,2
3,3,13,28}                                 -- peer group
n % 5 = 3 {18,8,23,3,13,28}
  4 |
{15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27,18,8,2
3,3,13,28,19,4,29,14,9,24}                 -- peer group
n % 5 = 4 {19,4,29,14,9,24}
  5 | {15,10,25,20,5,30}
```

```
                                                    -- peer
group n % 5 = 0 {15,10,25,20,5,30}
  6 | {15,10,25,20,5,30,16,6,11,1,21,26}
                                                    -- peer
group n % 5 = 1 {16,6,11,1,21,26}
  7 |
{15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27}
                                          -- ...etc
  8 |
{15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27,18,8,2
3,3,13,28}
  9 |
{15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27,18,8,2
3,3,13,28,19,4,29,14,9,24}
 10 | {15,10,25,20,5,30}
 11 | {15,10,25,20,5,30,16,6,11,1,21,26}
 12 | {15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27}
 13 |
{15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27,18,8,2
3,3,13,28}
 14 |
{15,10,25,20,5,30,16,6,11,1,21,26,12,7,17,22,2,27,18,8,2
3,3,13,28,19,4,29,14,9,24}
 15 | {15,10,25,20,5,30}
....
```

e.g. PARTITION BY only
PARTITION BY n % 5: 5 partitions + peer group NA

```
SELECT
  n, array_agg(n)
  OVER(PARTITION BY n % 5)  AS window
FROM t
ORDER BY n;

 n  |     window
════╪═══════════════════
  1 | {16,6,11,1,21,26}           -- partition n % 5 = 1
  2 | {12,7,17,22,2,27}           -- partition n % 5 = 2
  3 | {18,8,23,3,13,28}           -- partition n % 5 = 3
  4 | {19,4,29,14,9,24}           -- partition n % 5 = 4
  5 | {15,10,25,20,5,30}          -- partition n % 5 = 0
  6 | {16,6,11,1,21,26}
  7 | {12,7,17,22,2,27}
  8 | {18,8,23,3,13,28}
  9 | {19,4,29,14,9,24}
 10 | {15,10,25,20,5,30}
 11 | {16,6,11,1,21,26}
 12 | {12,7,17,22,2,27}
 13 | {18,8,23,3,13,28}
 14 | {19,4,29,14,9,24}
...
```

PARTITION BY n % 5 ORDER BY n: 5 partitions + **distinct peer** groups

```
SELECT
  n, array_agg(n)
  OVER(PARTITION BY n % 5 ORDER BY n) as window
FROM t
ORDER BY n;

 n  |     window
════╪════════════════════
  1 | {1}                          -- partition n % 5 = 1
peer {1}
  2 | {2}                          -- partition n % 5 = 2
peer {2}
  3 | {3}
  4 | {4}
  5 | {5}
  6 | {1,6}                        -- partition n % 5 = 1
peer {6}
  7 | {2,7}                        -- partition n % 5 = 2
peer {7}
  8 | {3,8}                        ...
  9 | {4,9}
 10 | {5,10}
 11 | {1,6,11}
 12 | {2,7,12}
 13 | {3,8,13}
 14 | {4,9,14}
 15 | {5,10,15}
 16 | {1,6,11,16}
 17 | {2,7,12,17}
 18 | {3,8,13,18}
 19 | {4,9,14,19}
 20 | {5,10,15,20}
...
```

```
-- WINDOW
SELECT
  rank() OVER w
FROM mytable
WINDOW w AS (PARITION BY n);
SELECT                             -- same inline
  rank() OVER(PARTITION BY n)
FROM mytable;

-- window + clauses
SELECT
FROM mytable
WHERE n % 10 = 0                   -- WHERE: before
WINDOW
```

```
WINDOW w AS (PARITION BY n)
ORDER BY n;                         -- ORDER BY: after
WINDOW


-- Window partitions
SELECT ...
WINDOW w AS ()            -- OK. No partition i.e. window
is whole table
WINDOW w AS (
  -- PARTITION BY
  PARTITION BY n
  PARTITION BY n % 10     -- expression
  PARTITION BY n, m       -- multiple

  -- ORDER BY
  ORDER BY n
  ORDER BY n % 3          -- expression
  ORDER BY n, m           -- multiple
)


-- ROW/GROUPS/RANGE: pick rows to include in window
--
SELECT ...
  WINDOW w AS (
  -- ROWS mode: count by row number
  --
  ROWS CURRENT ROW                    -- include current
row only
  ROWS 1 PRECEDING                    -- previous +
current row
  ROWS                                -- same
    BETWEEN 1 PRECEDING AND CURRENT ROW
  ROWS 1 FOLLOWING                    -- current + next
row
  ROWS 0 PRECEDING                    -- same as CURRENT
ROW
  ROWS 0 FOLLOWING
  ROWS -1 PRECEDING                   -- ERROR
  ROWS -1 FOLLOWING                   -- ERROR
  ROWS 100 PRECEDING                  -- same as
UNBOUNDED if exceeded
  ROWS 100 FOLLOWING
  ROWS UNBOUNDED PRECEDING
  ROWS UNBOUNDED FOLLOWING

  ROWS BETWEEEN
   5 PRECEDING AND 5 FOLLOWING        -- 5 previous upto
5 next
  ROWS BETWEEEN
   UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING             -- default for unordered
```

```
-- unordered
SELECT n, array_agg(n) OVER w AS window
FROM t
WINDOW w AS (PARTITION BY n % 3)
ORDER BY n;

-- same
SELECT n, array_agg(n) OVER w AS window
FROM t
WINDOW w AS (PARTITION BY n % 3 ROWS BETWEEN UNBOUNDED
PRECEDING AND UNBOUNDED FOLLOWING)
ORDER BY n;

 n  |             window
════╪═══════════════════════════════════
  1 | {1,2,3,4,5,6,7,8,9}
  2 | {1,2,3,4,5,6,7,8,9}
  ...
  9 | {1,2,3,4,5,6,7,8,9}
 10 | {10,11,12,13,14,15,16,17,18,19}
 11 | {10,11,12,13,14,15,16,17,18,19}
 ...
 19 | {10,11,12,13,14,15,16,17,18,19}
 20 | {20,21,22,23,24,25,26,27,28,29}
 21 | {20,21,22,23,24,25,26,27,28,29}
 ...
 50 | {50}
```

```
  -- GROUPS mode: count by peer groups
 GROUPS                                    -- ERROR.
Requires ORDER BY
 ORDER BY n
  GROUPS CURRENT ROW                       -- current
peer group only
  GROUPS 1 PRECEDING                       -- previous
peer + current peer
  GROUPS 1 FOLLOWING
  GROUPS UNBOUNDED PRECEDING
  GROUPS UNBOUNDED FOLLOWING
  GROUPS BETWEEN
   UNBOUNDED PRECEDING AND CURRENT ROW     -- default
for ordered
```

```
-- ordered
SELECT n, array_agg(n) OVER w AS window
FROM t
WINDOW w AS (PARTITION BY n % 2 ORDER BY n % 3)
ORDER BY n;

-- same
SELECT n, array_agg(n) OVER w AS window
FROM t
```

```
WINDOW w AS (PARTITION BY n % 2 ORDER BY n % 3 GROUPS
BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
ORDER BY n;


 n  |                    window
════╪═══════════════════════════════════════════════
  1 | {21,15,3,27,9,1,7,13,19,25}
  2 | {6,12,24,30,18,16,4,10,22,28,26,20,8,14,2}
  3 | {21,15,3,27,9}
  4 | {6,12,24,30,18,16,4,10,22,28}
  5 | {21,15,3,27,9,1,7,13,19,25,17,29,23,5,11}
  6 | {6,12,24,30,18}
  7 | {21,15,3,27,9,1,7,13,19,25}
  8 | {6,12,24,30,18,16,4,10,22,28,26,20,8,14,2}
  9 | {21,15,3,27,9}
 10 | {6,12,24,30,18,16,4,10,22,28}
 11 | {21,15,3,27,9,1,7,13,19,25,17,29,23,5,11}
 12 | {6,12,24,30,18}
 13 | {21,15,3,27,9,1,7,13,19,25}
...
```

```
 -- RANGE mode: count by value instead of row position
 RANGE
 ORDER BY n
  RANGE CURRENT ROW           -- current row only
  RANGE 1 PRECEDING           -- n - 1 upto current rows
  RANGE 5 PRECEDING           -- n - 5 upto current rows
  RANGE BETWEEN ...
)
```